

# Metaheurísticas aplicadas a la planificación de procesos de desarrollo

Autor: Miguel García Bohigues

# 1. Problema

## 1.1 Introducción

En esta memoria se presenta una solución para una instancia del problema de la asignación óptima de recursos. Concretamente, en esta aproximación se ha dotado de una temática bastante actual. En el campo del desarrollo software, la reactividad frente a los cambios de opinión del cliente o las novedades del mercado hace que, si deseamos que un producto sea viable, su desarrollo no deba ser muy largo.

Fruto de este entorno cambiante apareció la necesidad de realizar procesos más cortos de desarrollo en los que, de manera incremental, se va mejorando el producto, dotándole de más funcionalidades, hasta obtener una solución final. Este método permite también a las pequeñas y medianas empresas obtener financiación mientras trabajan en el producto.

Esto resulta en períodos de entre 1 y 3 meses en los que se ha de desarrollar la mejora necesaria o pactada. Para ello, normalmente el director de proyecto ha especificado y aproximado las tareas que será necesario realizar, el tiempo que costará cada una de ellas, y el grado de complejidad que presentan.

Lo que se propone con este proyecto es el desarrollo de una serie de algoritmos de búsqueda metaheurística de una solución óptima en función de las necesidades del cliente en cada una de las dos dimensiones que hemos enunciado previamente: apremio temporal o monetario.

Por lo tanto, nuestro objetivo con este problema es minimizar el tiempo de desarrollo de las tareas al mismo tiempo que intentamos mantener el coste tan bajo como podamos. No obstante, la importancia de cada uno de estos factores es configurable y la deberá marcar las necesidades del cliente.

## 1.2 Modelización

En esencia nuestro problema se compone por un conjunto finito de  $N$  tareas, cada una de estas con un perfil profesional que se ha pre-asignado y una aproximación temporal de las horas que le costará a un trabajador con dicho perfil realizar la tarea.

Por lo tanto, podemos disponer de los siguientes objetos con los que trabajar:

Tarea: {tiempo: Integer, rol: Sting}

Problema: tarea[]

Para el problema que hemos abordado, hemos considerado que la empresa en cuestión contrata los servicios de programación por horas a una consultora tecnológica que presenta 3 perfiles diferentes de programadores, aunque la inclusión o borrado de nuevos perfiles en el problema no aporta complejidad alguna.

Los roles considerados son: programador junior, programador senior y core. Enumerados de menor a mayor grado de experiencia. Como es de esperar, el coste monetario de cada uno de los roles es diferente e incremental con el grado de experiencia que presenta. En la siguiente tabla mostramos el sueldo anual para cada uno de los perfiles:

|         |         |         |
|---------|---------|---------|
| Junior  | Senior  | Core    |
| 19.000€ | 29.615€ | 37.953€ |

Estos salarios se han obtenido de la plataforma online [Glassdoor](#). No obstante, tal y como se ha comentado previamente, vamos a realizar una aproximación que solo contempla la contratación por horas. Por lo tanto, vamos a normalizar los datos presentados en la tabla anterior para reflejar el coste por horas que supondría para la empresa cada uno de los roles. Para ello hemos considerado 8h de sólo aquellos días que son laborables a lo largo del año (1888). Los resultados se muestran a continuación:

|           |           |           |
|-----------|-----------|-----------|
| Junior    | Senior    | Core      |
| 10.064€/h | 15.685€/h | 20.102€/h |

La experiencia de estos diferentes perfiles de trabajadores hace que puedan llevar a cabo ciertas tareas con mayor celeridad, disminuyendo el tiempo estimado inicial de éstas. La variabilidad en los tiempos puede verse en la siguiente tabla, dónde para cada uno de los perfiles requeridos para las tareas(filas) hemos establecido el factor que se le aplicará al tiempo de desarrollo en función del perfil profesional del trabajador que se le asigne(columnas). Un factor con un valor inferior a 1 indica que la tarea se realiza antes de lo estimado. Un valor superior, lo contrario.

| Rol de la tarea | Rol del trabajador |        |      |
|-----------------|--------------------|--------|------|
|                 | Junior             | Senior | Core |
| Junior          | 1                  | 0.85   | 0.7  |
| Senior          | 1.3                | 1      | 0.85 |
| Core            | 1000               | 1.3    | 1    |

Entendemos que por conocimientos y experiencia, un junior no puede realizar tareas pensadas para cores. Es por esto que el factor de multiplicación para este caso es 3 órdenes superior al resto, para indicar inviabilidad.

### 1.2.1 Modelización del genoma

Inicialmente consideramos utilizar una estructura formada por una lista de N elementos, siendo N el número de tareas a realizar. Cada uno de los elementos estaba representado por una tupla con el coste real de realizar la tarea y el rol que se le había asignado.

No obstante, esta idea se descartó rápidamente, puesto que en algoritmos como este que realizan la misma sección de código miles de veces, cuanto más plana sea la estructura de datos con la que trabaja mejor.

Realizando otra iteración sobre el proceso de modelado del problema, vimos que no era necesario almacenar el coste real, puesto que, si utilizábamos una estructura de datos externa para aplicar el factor, con sólo el rol podíamos tener la misma información y reducir la dimensionalidad del genoma.

Por ello, la aproximación final tiene en consideración sólo el rol asignado a la tarea, puesto que el resto de información puede ser computada en el momento de evaluar el fitness de cada uno de los individuos que componen la población, sin penalizar al tiempo de computo.

De esta manera, la codificación será una lista de N elementos, siendo N el número de tareas a realizar. Encada una de las posiciones del array tendremos una etiqueta que identificará cada uno de los posibles roles que podemos asignar. Para evitar la comparación de cadenas, vamos a utilizar un

único carácter para la representación. Típicamente, un individuo para un problema con 5 tareas podría ser el siguiente:

```
['S','C','J','C','C']
```

Por lo tanto, como la población se compone por un listado de genomas, trabajaremos con un array de listas como la anterior:

```
[['S','C','J','J','C'], ['S','S','J','C','C'], ['S','C','J','J','J'], ...]
```

Tal y como se puede deducir de lo expuesto anteriormente, la talla del problema aumenta linealmente tanto con el número de tareas a estimar, cómo con el número de individuos que conforman la población.

## 2. Desarrollo

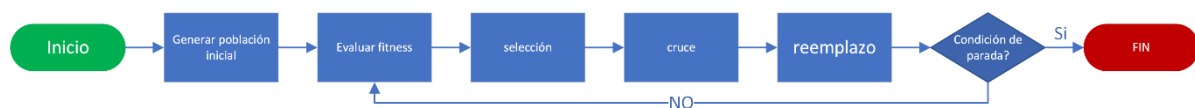
### 2.1 Lenguaje y aproximación

La implementación del algoritmo se ha realizado sobre el lenguaje Python. Aunque su rendimiento es claramente inferior delante de otros lenguajes como Java o C++, la flexibilidad y las diferentes librerías para el trato de listas y datos que ofrece han hecho que nos decantásemos por esta opción.

De las diferentes aproximaciones que se proponían para el desarrollo de los algoritmos, se ha optado por la implementación completa de los mismos. Esta variante nos ofrece mayor control y flexibilidad a la hora de parametrizar el modelo y variar su comportamiento para adaptarlo mejor al problema.

### 2.2 Algoritmo genético

El diagrama que se presenta a continuación se corresponde con la primera aproximación al desarrollo de este algoritmo. Tal y como se explicará en secciones futuras, se ha modificado el comportamiento de algunas fases además de haberse agregado funcionalidades nuevas. No obstante, pasamos a explicar cómo se ha desarrollado cada una de ellas:



#### Generación de la población inicial

Esta fase se ha parametrizado, pudiendo elegir el usuario qué cantidad de individuos desea tener en su población. La creación de individuos se ha hecho de manera aleatoria, para intentar maximizar la variabilidad y así ampliar el territorio de búsqueda.

#### Evaluación del fitness

Tal y como hemos mencionado en la introducción del problema, estamos frente a la minimización de dos parámetros diferentes. Esto hace que nuestro problema no tenga una única solución, sino una colección alineada en lo que se denomina “frontera de Pareto”. Por lo tanto, dejamos que sea el usuario el que decida cuál es la mejor decisión para la situación en la que se encuentra: Dar mayor importancia al tiempo o mayor importancia al dinero.

Para poder devolver una única solución para la importancia introducida por el usuario, hemos utilizado una proyección lineal para calcular el fitness de cada uno de los individuos de la población. Así, nuestra función de fitness es:

$$f(x) = \alpha * h + (1 - \alpha) * c$$

Siendo  $h$  el total de horas de desarrollo,  $c$  el coste total en € del desarrollo y  $\alpha$  el parámetro de proyección lineal que expresa la preferencia del cliente sobre priorizar menos horas o menos coste.

Entonces, nuestro objetivo es obtener ese espécimen que consigue minimizar el valor de  $f(x)$  para el valor de  $\alpha$  dado.

### **Selección**

Para la selección se ha decidido utilizar una selección por ruleta, otorgando mayor densidad de probabilidad a aquellos especímenes con mejor valor de  $f(x)$ . De esta manera, aunque primamos el cruce de especímenes buenos, dejamos cierta probabilidad para que los que no son tan buenos puedan ser también escogidos, dejando lugar a la exploración

### **Cruce y mutación**

Para generar nuevos individuos se ha optado por un cruce por segmentos. De esta manera, dados dos progenitores, se realizan un número parametrizable de particiones y éstas se intercambian generando así dos nuevos individuos como resultado.

Además, se ha proveído al sistema de una probabilidad de mutado, en la que se procederán a intercambiar dos posiciones aleatorias del genoma para así intentar evitar estancamientos.

### **Reemplazo**

El relevo generacional se ha realizado con una selección elitista de los individuos. Dependiendo del súper parámetro “numero de descendientes” se reemplazan los peores  $X$  individuos de la población por los hijos generados.

### **Criterio de parada**

El criterio de parada de nuestro algoritmo es puramente temporal. Transcurridos 10 minutos de ejecución el algoritmo se detiene devolviendo la mejor solución que ha encontrado. No obstante, como mejora se ha agregado un criterio de convergencia basado en las generaciones. Si transcurren 1500 generaciones sin que haya mejora en la solución, se detiene el proceso.

#### **2.2.1 Iteraciones y mejoras**

Antes de dar por válido el algoritmo anterior y pasar a la estimación de parámetros, se ha decidido testear el algoritmo propiamente. Para ello se han realizado 80 ejecuciones de un problema que constaba de 50 tareas y con un valor de  $\alpha = 0.65$ . En cada una de ellas se iba variando el valor de los macro-parámetros: población\_inicial, número\_descendientes, num\_particiones.

Estas ejecuciones han mostrado ciertas irregularidades con respecto a la solución que el sistema devolvía como resultado. Éstas eran muy variables y dependían en gran manera de la “suerte” que se tuviese en la población inicial a la hora de asignar escenarios inviables (juniors haciendo tareas de core). Tal y como se puede ver en la siguiente tabla:

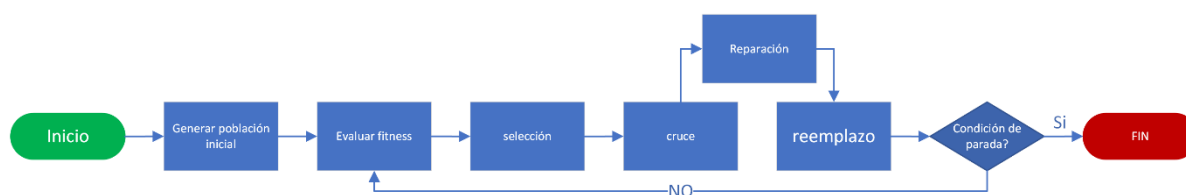
|                                    |                    |
|------------------------------------|--------------------|
| <b>Convergence rate:</b>           | 0.8375             |
| <b>Best solution:</b>              | 2672.9122537076273 |
| <b>Worst solution:</b>             | 8382.67486854      |
| <b>solution median:</b>            | 2685.82036992      |
| <b>Solution average:</b>           | 6730.60008592      |
| <b>Solutions far from optimal:</b> | 9                  |
| <b>Time Average:</b>               | 20.40s             |
| <b>Time median:</b>                | 16.21s             |

Como vemos, existe una gran diferencia entre la mejor solución arrojada por el algoritmo y la peor. Además, la media se aleja bastante del valor óptimo. Para evitar esta aleatoriedad y falta de convergencia, hemos mejorado el algoritmo, introduciendo un proceso de corrección de los individuos “erróneos”, explicado en la siguiente subsección.

### Primera mejora: reparación de individuos

Hasta el momento, el algoritmo permitía la existencia de individuos no factibles (aquellos que asocian tareas propias de un Core a un Junior) pero con probabilidad muy baja para reproducirse. Al verse que esto afectaba al determinismo del sistema a la hora de obtener la mejor solución, se ha decidido repararlos, sustituyendo esta asignación por una asignación arbitraria de un rol que sí puede realizar estas tareas (Senior o Core)

Así, el diagrama de flujo del algoritmo queda como sigue:



Tras hacer esta modificación en el modelo, hemos vuelto a lanzar las mismas pruebas, obteniendo en este caso unos resultados más homogéneos para los diferentes parámetros. Tal y como se muestra en la tabla siguiente:

|                                    |               |
|------------------------------------|---------------|
| <b>Convergence rate:</b>           | 0.8375        |
| <b>Best solution:</b>              | 2672.73969341 |
| <b>Worst solution:</b>             | 2739.16795068 |
| <b>solution median:</b>            | 2683.61590108 |
| <b>Solution average:</b>           | 2696.03487598 |
| <b>Solutions far from optimal:</b> | 0             |
| <b>Time Average:</b>               | 19.18s        |
| <b>Time median:</b>                | 13.92s        |

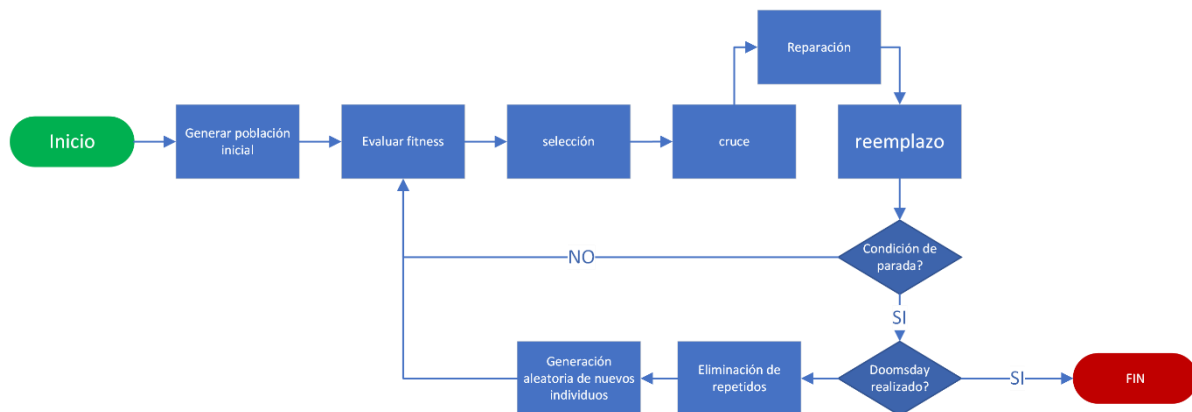
Como vemos, el determinismo del algoritmo ha mejorado bastante, reflejando en cada ejecución soluciones que son válidas y cercanas a las existentes en la frontera de Pareto. A pesar de esta mejora, hemos querido ajustar aún más el algoritmo. En las iteraciones, hemos observado que hasta en 16 de las 80 iteraciones se ha llegado a un criterio de convergencia sobre una solución encontrada **antes de la 10ª generación** y que era mejorable. Para ajustar más, introducimos al algoritmo un proceso extra que hemos denominado “día del juicio final”

## Segunda mejora: Día del juicio final

Observando las variaciones más de cerca, vemos que se produce por la aportación de soluciones precisas pero mejorables, en momentos tempranos de la ejecución. Esto hacía que el criterio de parada detectase un estancamiento en la mejora de los fitness y diese por finalizada la ejecución cuando aún era posible la mejora.

Para evitar esta convergencia temprana, se pensó en realizar un proceso de reemplazo catastrófico, lo que hemos denominado “día del juicio final”. En este proceso, lo que hacemos es eliminar los especímenes repetidos de la población y generar nuevos de manera aleatoria. Así conseguimos movernos dentro del espacio de soluciones y aproximar mejor la solución.

Este proceso de eliminación de individuos se realiza la primera vez que se llega a un punto de convergencia y sólo se realiza una vez por ejecución. Con esta modificación, el flujo del algoritmo queda tal y como se ve en la imagen:



En la siguiente tabla se muestran los resultados obtenidos tras esta modificación en el algoritmo:

|                                    |               |
|------------------------------------|---------------|
| <b>Convergence rate:</b>           | 0.9           |
| <b>Best solution:</b>              | 2672.65772727 |
| <b>Worst solution:</b>             | 2720.98625160 |
| <b>solution median:</b>            | 2674.44390613 |
| <b>Solution average:</b>           | 2676.83755145 |
| <b>Solutions far from optimal:</b> | 0             |
| <b>Time Average:</b>               | 52.85s        |
| <b>Time median:</b>                | 38.47s        |

Como se puede apreciar, los tiempos medios de ejecución aumentan fruto de este nuevo criterio de convergencia, pero las soluciones generadas presentan una variabilidad mínima, asegurando que el algoritmo en cualquier iteración aportará una solución válida.

### 2.2.2 Estimación de parámetros

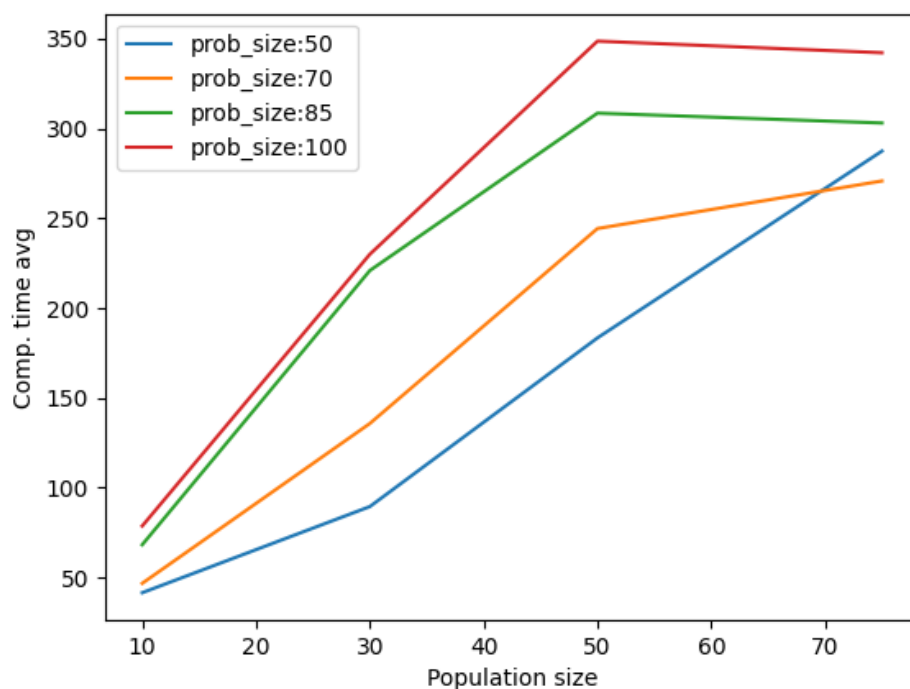
Derivado de la implementación que hemos realizado del algoritmo genético, disponemos de 3 macro-parámetros a estimar: **El tamaño de la población**, **el número de descendientes** en cada generación y **el número de particiones** a realizar para cruzar los especímenes.

Para estimar estos parámetros hemos realizado diversas iteraciones con problemas de diferentes tallas. En concreto, las tallas varían desde las 50 tareas hasta las 100, pasando por

dos puntos intermedios en 70 y 85 tareas. Se han realizado 80 ejecuciones con diferentes combinaciones para cada una de las tallas, un total de 320 ejecuciones.

De los tres parámetros mencionados, vamos a empezar estimando el tamaño de la población con la que trabajar. Para estimar este parámetro se ha de tener en cuenta el tiempo de cómputo y la precisión de la solución devuelta por el algoritmo. Una población mayor puede proporcionar una mayor cantidad de individuos buenos, pero también catapultar el tiempo de cómputo.

En la siguiente imagen, vemos como aumenta el tiempo medio de cómputo de las ejecuciones a medida que aumentamos el tamaño de la población para cada una de las instancias del problema.

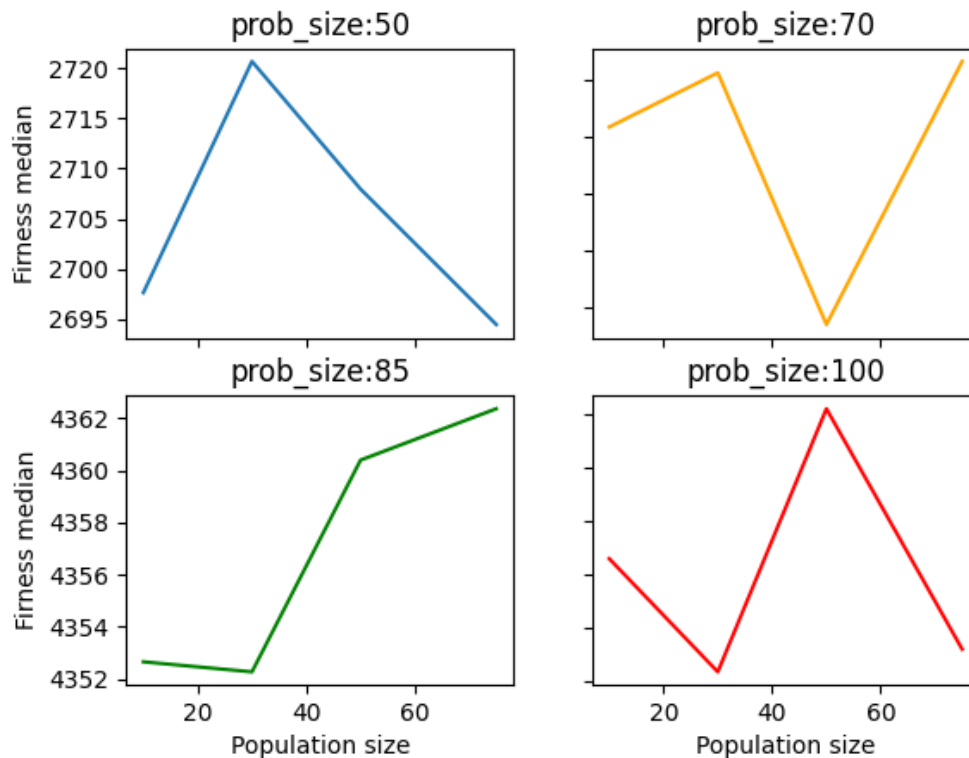


Como era de esperar, el tiempo medio de cómputo aumenta a medida que aumenta el número de individuos en la población, bien sea por una convergencia más lenta, o por un coste mayor de la función de evaluación.

Teniendo en cuenta el incremento temporal que supone aumentar la población inicial (hasta 7 veces más tiempo de cómputo) vamos a observar cómo afecta a la solución propuesta, para ver si efectivamente es beneficioso aceptar el incremento de tiempo o no.



En la siguiente gráfica podemos observar la mediana de las diferentes ejecuciones que hemos realizado sobre los diferentes tamaños de población. Allí podemos observar como no hay un valor de la población que minimice claramente la salida para todos los problemas, más bien vemos que el resultado es heterogéneo, no siendo ni monótonamente creciente ni decreciente.



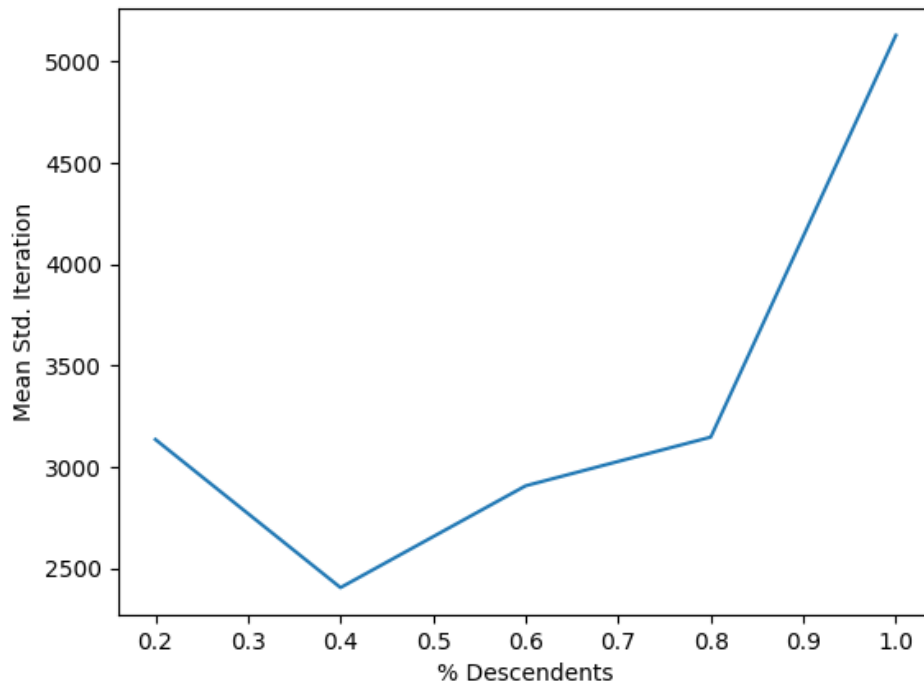
La falta de convergencia en los resultados para las diferentes poblaciones hace que no podamos decidir una de manera certera. No obstante, tal y como se puede ver en la siguiente tabla, el diseño del algoritmo hace que la variabilidad de los resultados entre las diferentes ejecuciones sea de un orden prácticamente despreciable.

| Talla del problema | Variación del fitness |
|--------------------|-----------------------|
| 50                 | ±1.25%                |
| 70                 | ±1.19%                |
| 85                 | ±1.36%                |
| 100                | ±1%                   |

Como vemos, la variabilidad entre las ejecuciones es mínima, permitiéndonos escoger una población inicial que minimice el tiempo de cómputo, ya que sabemos que la solución, aunque pueda ser mejorable, no estará lejos de la óptima. Por lo tanto, establecemos **tamaño\_población = 10** como el valor idóneo para nuestro problema.

Con el tamaño de la población ya definido, podemos pasar a estimar el siguiente parámetro: el número de descendientes a generar. Este parámetro lo hemos configurado en forma de "el porcentaje de individuos que son reemplazados en cada iteración". Para estimarlo

hemos realizado varias pruebas reemplazando un 20%, 40%, 60%, 80% y la población íntegra en cada generación. Con este parámetro lo que nos interesa es ofrecer suficiente variabilidad en la población para poder explorar nuevas soluciones, pero sin generar una variabilidad que nos dificulte llegar a la solución óptima.



Para evaluar esto se ha calculado la desviación típica de las iteraciones necesarias para llegar a la solución para cada porcentaje de reemplazo por parte de los descendientes. Posteriormente, para homogeneizar la gráfica, se ha calculado la media para cada uno de los problemas, dando como resultado la gráfica que se muestra justo arriba. En ella podemos observar como el valor que, de media, presenta menos desviación típica a lo largo de todos los problemas es un **reemplazo del 40% de la población**. Mientras que, coherentemente, el valor que más desviación presenta es el reemplazo de la población íntegramente.

Finalmente, hemos de obtener el mejor valor para nuestro último super-parámetro: el numero de particiones a realizar cuando hacemos el cruzado. Este valor se ha de ajustar a la longitud de los genomas, por lo que depende de la talla del problema (número de tareas). El numero de particiones lo hemos representado con un porcentaje, de igual manera que el numero de descendientes. En el siguiente ejemplo se muestra el comportamiento para un 20% de particiones y un problema con 10 tareas:

$$N_{\text{particiones}} = 10 * 0.2 = 2$$

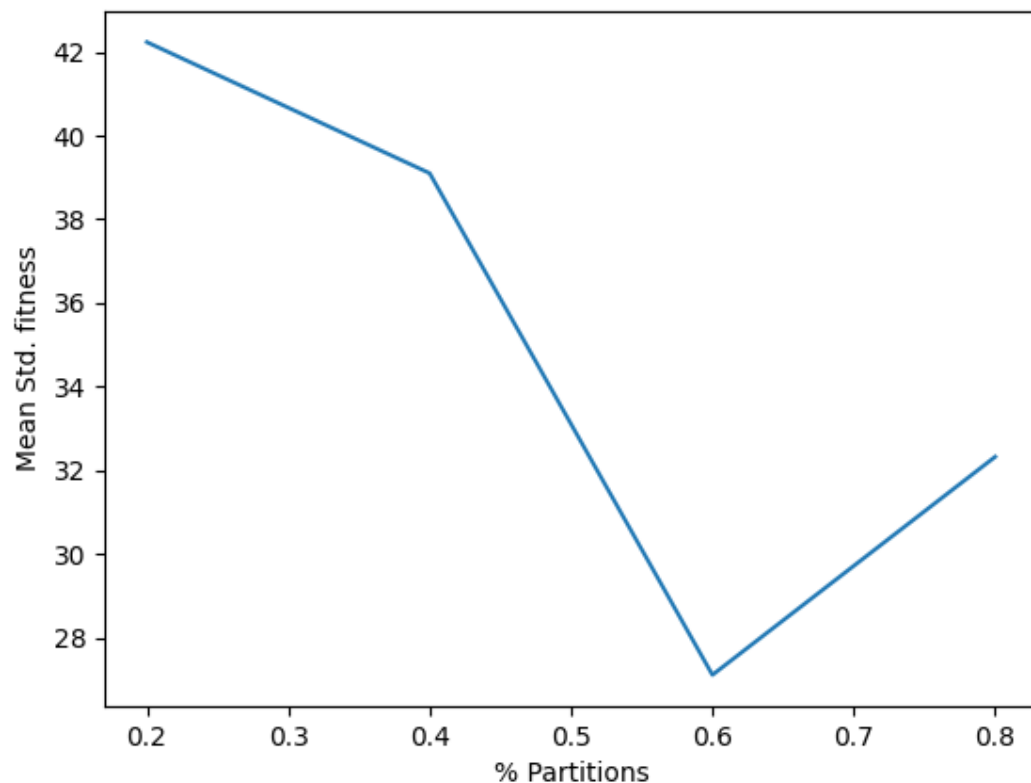
P1: ['J','J','C','S','J','C','C','S','S','S']

P1\_particionado: ['J','J','C'] ['S','J','C','C'] ['S','S','S']

Un alto número de particiones puede llevarnos a mayor variabilidad en la solución debido al mayor número de intercambios realizados en los genes, mientras que un valor más

reducido de las particiones puede llevarlos a estancamientos rápidos en la búsqueda de la solución.

Para estimar el mejor valor hemos seguido un camino similar al escogido para analizar el porcentaje de descendientes: realizar la media para cada problema de la desviación típica del mejor fitness obtenido por cada uno de los porcentajes de partición. EN la siguiente gráfica se pueden observar los resultados.



Como podemos ver, con diferencia el valor más determinista a la hora de devolver una solución optima es aquel que utiliza **talla\*0.6 como numero de particiones**.

Con todo esto, nuestro algoritmo ha mostrado que los mejores valores para los super-parámetros son:

| Parámetro       | Valor |
|-----------------|-------|
| Población       | 10    |
| % descendientes | 0.4   |
| % Particiones   | 0.6   |

## 2.3 Enfriamiento Simulado

El algoritmo de enfriamiento simulado lo hemos desarrollado siguiendo las mismas directrices que el algoritmo genético. En el siguiente diagrama se muestran las fases que corresponden a la implementación inicial del mismo.



### Generación del individuo inicial

Para la generación del individuo inicial se ha utilizado el mismo mecanismo que el utilizado en el genético, con la única modificación que, en este caso, se ha decidió implementar la reparación de individuos desde el principio. Vista su clara mejora en los resultados anteriores.

### Evaluación del fitness

La evaluación del fitness no se ha visto modificada. Se ha utilizado la misma función que en el algoritmo genético. Simplemente se ha realizado una inversión de su valor para que se adapte a los criterios matemáticos del cálculo de la probabilidad de selección. Pero repetimos, no se ha modificado su evaluación, la formula se mantiene la misma, sólo se ajusta el valor para no generar valores de “e” muy elevados. Puesto que si no la invertimos vemos que toma valores 3 magnitudes inferiores a 1 (0.00X)

### Generación del vecino

Para la generación de los vecinos, al principio se optó por utilizar el método de intercambio implementado en el algoritmo genético. Este método, aunque válido, tardaba mucho en converger, llegando a puntos en los que se terminaba la ejecución por convergencia forzada teniendo como mejor solución una que no era óptima.

Por ello se decidió modificar esta generación para dotarla de más variabilidad, manteniendo la búsqueda local. La función de generación recorre el individuo actual y para cada gen dentro de su genoma, aplica una probabilidad de mutación. En caso de mutar, ese gen se cambia por otro válido. Para asegurar que en todas las iteraciones se genera al menos una mutación, cada vez que se “falla” al mutar, se incrementa la probabilidad de mutado del siguiente gen.

### Decremento de la temperatura

Para el decremento de la temperatura, se ha optado por el uso de la siguiente función:

$$\frac{T}{1+k*T}$$

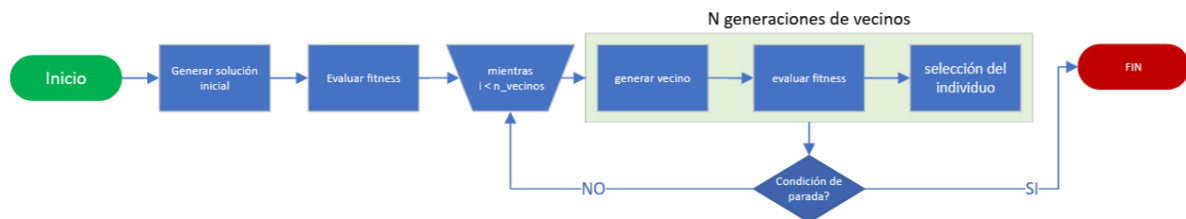
Dónde  $T$  es la temperatura y  $k$  el valor de decremento de la misma. Esta ecuación introduce los dos primeros hiperparámetros del algoritmo.

### Límite de convergencia forzado

De igual manera que hicimos con el algoritmo genético, hemos optado por un límite de convergencia forzado representado por un número de iteraciones en las que no hemos obtenido mejora. Para el caso del enfriamiento simulado, como sus iteraciones son bastante menos costosas, hemos optado por un valor de 15000 después de probar varios valores.

### 2.3.1 Iteraciones y mejoras

Como se puede observar, por cada iteración se genera un solo individuo y se comparan los fitness. Esta aproximación, aunque válida, como pasase con los algoritmos genéticos, mostraba bastante variabilidad a la hora de arrojar la mejor solución.



Para intentar resolver este problema, la aproximación que se ha abordado es la de la generación de varios individuos en la misma iteración. De esta manera se estudian varias posibles soluciones a la vez dentro del espacio y se puede avanzar más y mejor en la búsqueda.

Como vemos en la figura, no se ha realizado ninguna modificación sobre el proceso de generación, simplemente se itera sobre la solución ya propuesta para el número de vecinos que se haya configurado, otro hiperparámetro.

### 2.3.2 Estimación de parámetros

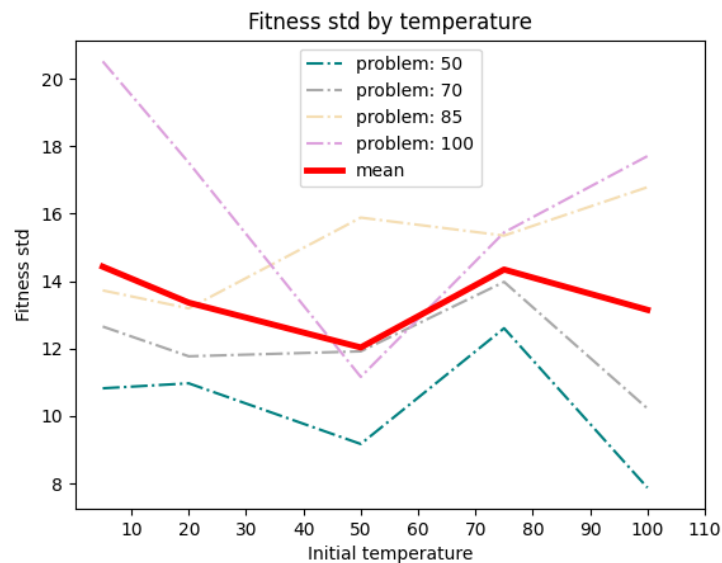
Tal y como se ha introducido en la sección anterior, para el algoritmo de enfriamiento simulado disponemos de tres parámetros a estimar: la **temperatura**, el valor de reducción **k**, y el **número de vecinos** a generar.

El proceso de ejecución se ha mantenido el del algoritmo genético, diversas iteraciones con diversos valores para los diferentes tamaños de problema. En este caso se han realizado 100 ejecuciones para diferentes valores de los parámetros a estimar y para cada uno de los problemas propuestos. Un total de 400 iteraciones.

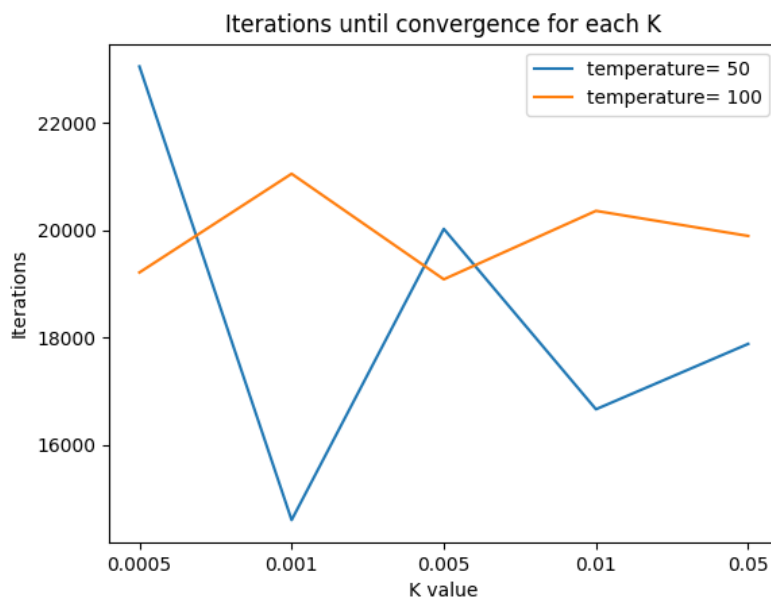
Como veremos en siguientes figuras, la variabilidad de los resultados hemos conseguido que no sea muy elevada, siendo todas del mismo orden y fluctuando con un  $\pm 50$  respecto al valor medio del fitness (2730 para  $p=50$ , 4350 para  $p=75$ , 4400 para  $p=85$  y 5500 para  $p=100$ )

Por ello, para estimar nuestro primer parámetro, la temperatura, nos interesa encontrar aquel valor que minimice el valor de nuestra función de fitness y que al mismo tiempo sea más consistente con sus resultados. En la siguiente imagen se puede observar la desviación típica obtenida sobre cada uno de los problemas para los diferentes valores de temperatura que hemos introducido. En una primera representación no había un valor que fuese candidato claro, puesto que como podemos observar, para tallas más reducidas, el valor extremo  $t=100$  ha funcionado el mejor, pero para tallas más elevadas ha creado más divergencia. Por ello, se añadió al diagrama la media a través de todos los problemas,

pudiendo observar que el valor  $T=50$  es el que minimiza esta variación para todos los problemas, en general. Seguido muy de cerca por  $T=100$ .



No siendo capaces de determinar exactamente qué valor de la temperatura seleccionar, vamos a arrastrar esta decisión a la estimación del parámetro  $K$ . Éste está íntimamente relacionado con la temperatura, por lo que estimar su valor puede arrojar un poco de luz sobre su valor idóneo. Además, grandes unidades de  $K$  hacen que  $T$  decrezca muy rápido, perdiendo capacidad de exploración, mientras que valores muy pequeños hacen que haya más variabilidad y por lo tanto se tarde más en converger. Es por esto por lo que para determinar el valor de  $K$  hemos contrapuesto el número de iteraciones que se ha necesitado para llegar a la convergencia para cada uno de los valores de  $K$  y de temperatura.

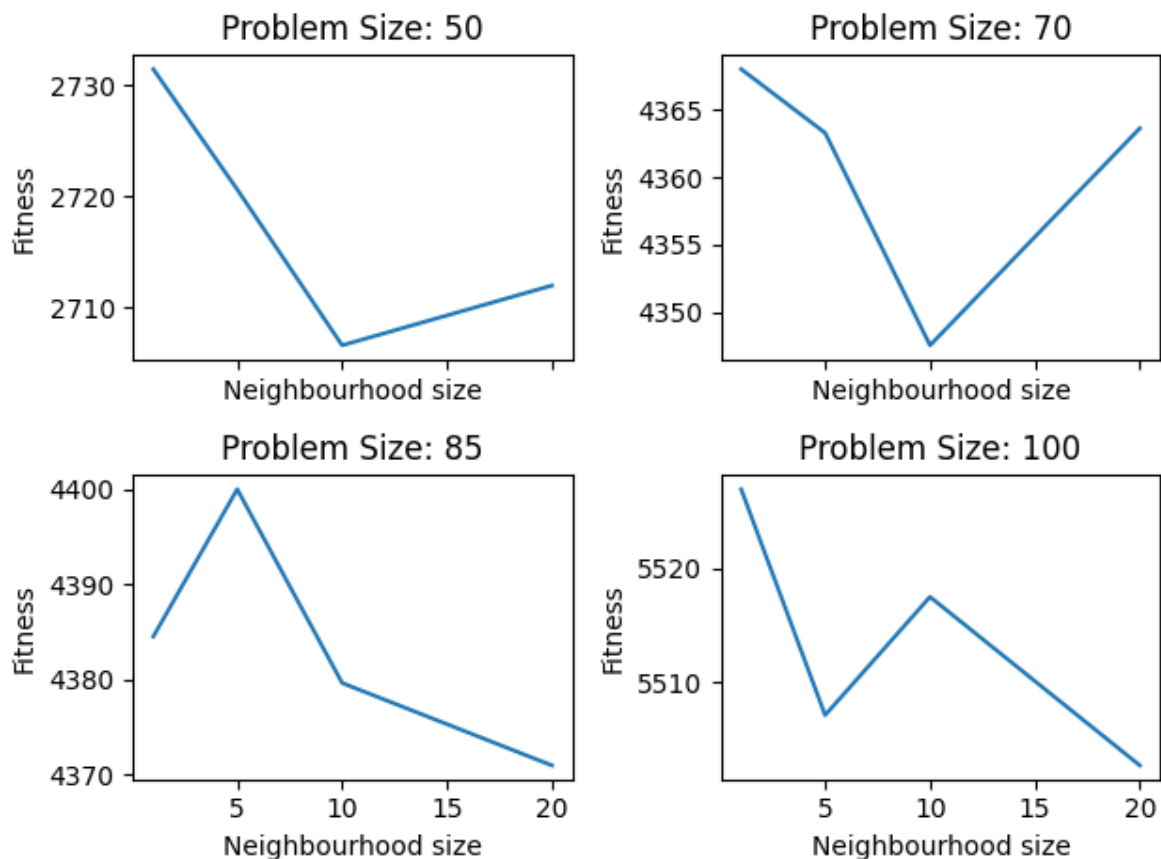


Como vemos en la figura, el valor de la  $K$  para  $T=100$  no parece afectar mucho a las iteraciones necesarias para converger. Siendo que su fluctuación ronda las 1000 iteraciones, que en el caso de nuestro algoritmo de enfriamiento simulado trata de un tiempo  $<5s$ .

Por otro lado, vemos que, de media para nuestros problemas, un valor de  $K=0.001$  reduce en 2000 al segundo mejor valor y en **7000** al mayor. Este cambio sí es significativo, cosa que nos hace decantarnos por unos valores de  $K=0.001$  y  $T=50$

Finalmente, hemos de estimar el mejor valor para el tamaño de nuestro vecindario de soluciones. Para hacer esto se han filtrado los resultados por los parámetros estimados hasta el momento. Como vemos en la imagen que se muestra a continuación, y de igual manera que pasaba con la temperatura, no exista un valor claro de  $N$  que nos minimice el valor de nuestro fitness. No obstante, sí que observamos que las variaciones son mínimas, por lo que podemos optar por seleccionar aquel valor de  $N$  que mejor dato general arroje.

Como vemos es  $N=20$ , puesto que para tallas grandes minimiza el fitness, para la talla 50, presenta prácticamente el mismo valor medio que su menor valor, y aunque para talla 70 se dispara podemos achacarlo a la variabilidad mencionada previamente, puesto que para talla 85, muy similar, ha funcionado correctamente.



| Parámetro    | Valor |
|--------------|-------|
| Temperatura  | 50    |
| K            | 0.001 |
| Núm. vecinos | 20    |

### 3. Comparativa y Conclusiones

Llegamos a la parte más interesante del proyecto, la comparativa de funcionamiento entre los diferentes algoritmos realizados y optimizados para la tarea. En esta sección se van a evaluar las prestaciones de ambos sistemas en función de métricas válidas para comparar.

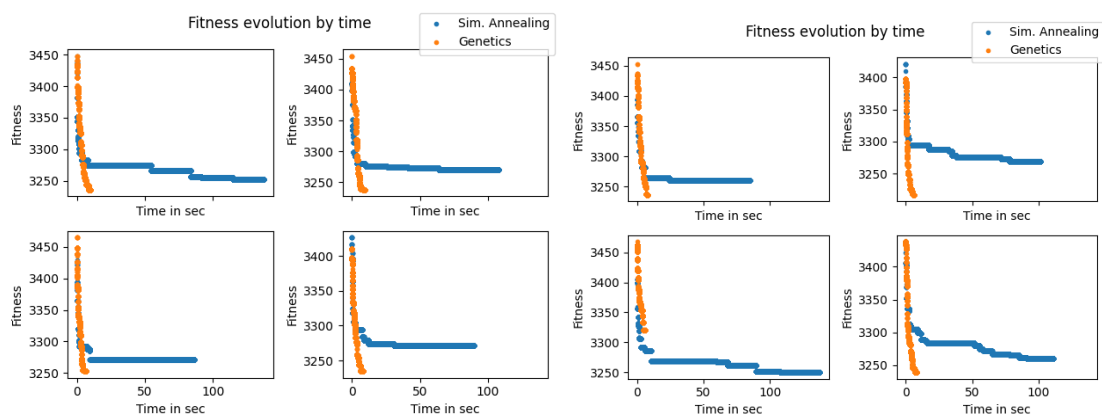
- La primera de estas va a ser la velocidad del sistema para viajar de la solución inicial a la solución que devuelva como optima al final de su ejecución.
- La segunda comparativa que vamos a realizar va a ser la variabilidad o consistencia que presentan sus soluciones
- Y la última será la proximidad a la solución óptima para el problema

Para realizar esta comparativa, vamos a generar un nuevo problema, que constará en este caso de 65 tareas. Estas tareas, al igual que se ha hecho con los problemas anteriores, tendrán estimaciones aleatorias y roles asignados aleatorios. Para él calcularemos la solución óptima y veremos cómo se comportan ambos algoritmos a lo largo de 8 ejecuciones de cada uno.

Para tener un baseline con el que poder comparar los resultados, hemos calculado del valor del fitness para la asignación trivial: para cada tarea asignar justo el rol que necesita. Sin intentar ninguna optimización. Los resultados de este baseline se muestran en la siguiente tabla.

| Parámetro           | Valor     |
|---------------------|-----------|
| Horas de desarrollo | 566       |
| Coste total (€)     | 8463.15 € |
| Valor del fitness   | 3330.00   |

Tal y como se ha mencionado previamente, el primero de los aspectos a estudiar va a ser la velocidad de convergencia de los diferentes sistemas para la talla del problema establecido. En las siguientes gráficas podemos ver los resultados de las 8 iteraciones realizadas

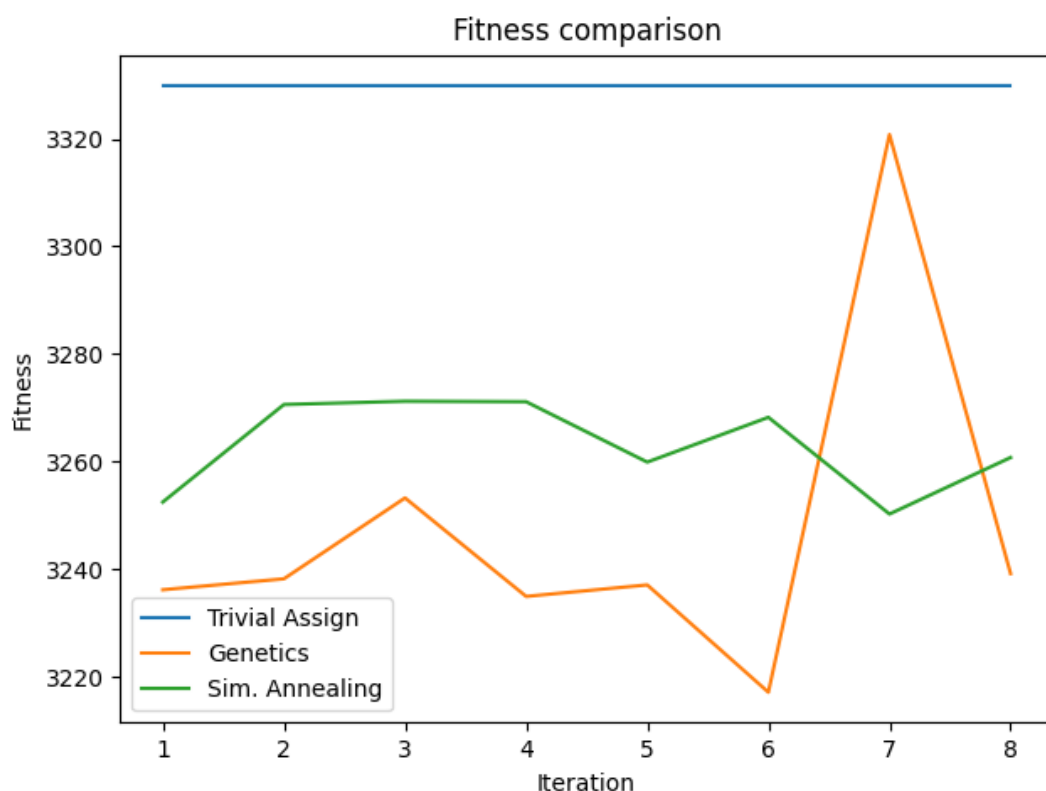




Como se puede observar, prácticamente siempre el algoritmo genético converge en menos de 10 segundos, mientras que el algoritmo de enfriamiento simulado requiere de hasta 10 veces más segundos para obtener su solución óptima por convergencia. Esto puede ser dado a la limitación de iteraciones para converger, aunque vemos que, a pesar de este tiempo extra de ejecución, no es capaz de generar soluciones mejores que el algoritmo genético. Concluimos en este aspecto que, el algoritmo genético para nuestro problema no solo proporciona mejores soluciones en la mayoría de los casos, sino que también las genera en menos tiempo.

El segundo elemento a analizar es la consistencia en la proposición de soluciones. Para evaluar esto, vamos a tener en cuenta la desviación típica del valor del fitness arrojado por las diferentes ejecuciones para cada uno de los algoritmos. Los resultados han sido de  $std_{gen} = 29.3444$  y de  $std_{Sim.An} = 7.9648$ . Como vemos, el algoritmo de enfriamiento simulado es más consistente con sus resultados, presentando menor variabilidad. Sin embargo, remitiéndonos a los resultados del estudio anterior, a pesar de que los resultados sean más consistentes, también son peores y más tardíos.

Finalmente, para reflejar los resultados obtenidos respecto al fitness, compactados en una sola gráfica, vemos la siguiente imagen. En ella se muestran los resultados del algoritmo de enfriamiento simulado, del genético y de la solución trivial.



Con esto concluimos nuestro estudio de aplicación de técnicas metaheurísticas al problema propuesto. El algoritmo genético ha mostrado un rendimiento por encima del enfriamiento

simulado y es, por tanto, con la implementación realizada, el mejor para el problema que tenemos entre manos.